

COBRIA: contratos reconstruibles y verificables para sistemas NoSQL documentales

Billy Jak Cordero Porras
Investigador independiente, Coto Brus, Costa Rica

Agosto de 2026

Resumen

La flexibilidad de los sistemas NoSQL documentales permite adaptar representaciones a sus patrones de acceso, pero también puede producir copias derivadas sin autoridad, aislamiento o reparación verificable. Este artículo presenta COBRIA, una síntesis arquitectónica normativa que reúne fuente canónica, transformación versionada, ámbito obligatorio, frescura, equivalencia, reconstrucción y responsabilidad operacional. Core 1.0 se mantiene congelado; Core 1.1 añade requisitos trazables, modelo de amenazas y núcleo independiente del proveedor; Core 1.2 incorpora un caso de referencia, hipótesis refutables, banco de fallos y protocolos de revisión y transferencia. La evaluación comprende 180 corridas embebidas con PouchDB y LokiJS, 270 corridas distribuidas con MongoDB 8.0, CouchDB 3.4 y OpenSearch 2.19.3, 450 corridas de líneas base, 300 inyecciones de fallos y 30 ensayos locales de concurrencia y recursos. P1, P2, R1, S1 y A1 se superaron conjuntamente: equivalencia lógica completa, reconstrucción idempotente, rechazo sin ámbito, cero documentos cruzados y manifiestos reproducibles. OpenSearch 3.2.0 no produjo cifras por una incompatibilidad de Java 24 bajo emulación x86_64; la sustitución 2.19.3 se declara explícitamente.

Palabras clave: NoSQL documental; datos canónicos; proyecciones reconstruibles; multiinquilinato; procedencia; inteligencia artificial.

1. Introducción

Una aplicación documental rara vez conserva una sola forma de cada hecho. Las reglas necesitan documentos completos; las interfaces, respuestas estrechas; la analítica, cortes estables; y la inteligencia artificial, fragmentos con evidencia y autorización. Duplicar puede ser correcto, pero una copia se vuelve peligrosa cuando nadie puede explicar de dónde provino, qué versión representa, a qué ámbito pertenece o cómo se repara.

COBRIA no reclama como nuevos la separación de lectura y escritura, las vistas materializadas, los eventos, la procedencia o los productos de datos. Tampoco es una estructura de datos, un motor o un ORM. Su contribución es composicional y normativa: hacer que autoridad, ámbito, equivalencia, reconstrucción y retiro acompañen a cada representación derivada. La pregunta es: *¿qué contrato permite optimizar lecturas documentales sin crear una autoridad paralela ni debilitar aislamiento, trazabilidad y reparación?*

Las contribuciones son: (1) una especificación congelada y una extensión compatible; (2) quince requisitos trazables y tres niveles de conformidad; (3) una implementación de referencia desacoplada del proveedor; (4) cuatro adaptadores NoSQL, dos evaluados y dos preparados; (5) pruebas ejecutables y un modelo de amenazas; y (6) una réplica local con resultados generados automáticamente.

2. Trabajo relacionado y vacío

Bigtable mostró la relación entre forma de almacenamiento y patrones de acceso [2]; Dynamo formalizó decisiones de partición, disponibilidad y consistencia [3]. Las vistas materializadas e incrementales explican

cómo mantener resultados derivados [5]; CQRS separa modelos de lectura y escritura [4]; y la procedencia describe el origen de resultados [1].

La recuperación aumentada combina memoria paramétrica y no paramétrica y reconoce la procedencia como un problema relevante [6]. En sistemas multiinquilino, la documentación de MongoDB exige que las consultas sobre colecciones compartidas incluyan el identificador del inquilino y advierte que la segmentación lógica debe aplicarse en la aplicación [7]. COBRIA convierte esas preocupaciones en obligaciones verificables que atraviesan documento, proyección, caché, recuperación y evidencia.

Estos trabajos no obligan, en una sola ficha verificable, a declarar simultáneamente autoridad, ámbito, equivalencia, reconstrucción, frescura, propiedad y retiro. COBRIA cubre ese vacío de integración sin presentar sus componentes como invenciones aisladas.

3. Especificación COBRIA Core 1.0

Sea C una colección canónica. Una proyección P_i se describe por:

$$K_i = \langle C, f_i, \theta_i, s, v, \phi_i, e_i, r_i, o_i \rangle,$$

donde f_i es la transformación; θ_i , catálogos y políticas; s , ámbito; v , versión; ϕ_i , frescura; e_i , equivalencia; r_i , reconstrucción; y o_i , propiedad operacional.

La especificación exige diez invariantes: autoridad canónica, ámbito obligatorio, procedencia, reconstrucción, equivalencia, idempotencia, revisión monotónica, publicación segura, operabilidad y fallo cerrado. Una candidata atraviesa los estados propuesta, construcción, verificación, activa y retirada. Una construcción fallida no modifica la versión activa.

3.1. Niveles de conformidad

Cuadro 1: Niveles normativos de COBRIA Core 1.0.

Nivel	Obligaciones
C1 Fundamental	Autoridad, ámbito, identidad estable, revisión y frontera de escritura.
C2 Reconstruible	C1 más transformación versionada, idempotencia, equivalencia, reconstrucción y publicación segura.
C3 Gobernado	C2 más procedencia, frescura, observabilidad, responsable y retiro.

3.2. Extensión Core 1.1

Core 1.1 no reescribe la línea base. Asigna identificadores estables a las obligaciones (por ejemplo, COBRIA-AUT-001, COBRIA-SCP-001 y COBRIA-REC-001), enlaza cada requisito con una prueba, un ejemplo y un contraejemplo, y separa cinco capas: núcleo, patrones, mecanismos, método de adopción e implementaciones. Esta separación evita confundir una recomendación pedagógica con una condición de conformidad.

4. Modelo de amenazas

El análisis considera confusión entre organizaciones, edición directa de derivados, repetición de eventos, regresión de revisiones, manipulación de proyecciones, fallos durante publicación, envenenamiento analítico, recuperación vectorial no autorizada, evidencia insuficiente, herramientas de agentes excesivas, eliminación incompleta y dependencia del proveedor. Los controles COBRIA se prueban en las fronteras de entrada, proyección, ámbito, recuperación y publicación. El modelo no sustituye autenticación, cifrado, respaldo ni consenso del motor.

5. Método

La investigación sigue ciencia del diseño: identificación, objetivos, construcción, demostración, evaluación y comunicación. La implementación neutral se adaptó a PouchDB 9.0.0 y LokiJS 1.5.12. Son motores documentales embebidos con modelos de persistencia distintos; permiten replicación local, pero no sustituyen servicios administrados.

Se ejecutaron tres escalas (100, 1,000 y 5,000 documentos), dos motores y treinta corridas por celda, para 180 corridas. Cada documento incluye identidad, ámbito, estado, valor, revisión y tiempo de evento. La corrida completa es la unidad informada. Los resultados crudos se conservan como JSONL y los resúmenes como JSON, CSV, Markdown y macros de $\text{\LaTeX}$. No se eliminaron valores atípicos; se añadieron p95, p99, desviación absoluta mediana e intervalos bootstrap del 95 % con 4,000 remuestras y semilla registrada.

MongoDB, CouchDB y OpenSearch fueron implementados mediante una interfaz común. El primero recibe una colección del controlador oficial por inyección; los otros dos usan API HTTP. La réplica distribuida usa operaciones por lotes nativas para carga y reconstrucción, mientras P2 conserva dos escrituras individuales observables. Se ejecutaron tres escalas y treinta corridas por celda, 270 corridas adicionales. Los servicios se ejecutaron secuencialmente en Colima x86_64 con dos CPU y aproximadamente 3 GB de RAM para evitar interferencia por presión de memoria.

Cuadro 2: Pruebas normativas.

Código	Propiedad	Aceptación
P1	Lectura canónica frente a proyectada	Resultado lógico idéntico.
P2	Amplificación de escritura	Operaciones y bytes observables.
R1	Reconstrucción total	Equivalencia e idempotencia.
S1	Aislamiento por ámbito	Rechazo sin ámbito y cero cruces.
A1	Producto analítico	Manifiesto, procedencia, partición y abstención.

5.1. Hipótesis comparativas

La extensión 1.2 formula ocho hipótesis. H1 espera menos derivados sin procedencia; H2, menor recuperación manual tras pérdida; H3, cero cruces bajo consultas adversariales; H4 anticipa mayor costo de escritura; H5 evalúa si un equipo externo implementa C2 solo con la especificación; H6 espera eliminar la ventana vacía mediante publicación por versión; H7 exige reproducir conjuntos por huella; y H8 espera cero candidatos ajenos cuando el ámbito precede a la similitud. H5 permanece sin evaluar porque requiere participantes independientes.

5.2. Caso de referencia

El caso neutral registra observaciones ficticias de fauna. Un expediente canónico con identidad, ámbito, revisión y versión alimenta un índice de búsqueda. Se elimina el índice, se construye una candidata desde documentos y catálogo congelados, se compara equivalencia y se publica mediante alias. El mismo caso extiende el contrato a un corte analítico temporal y a recuperación para IA con abstención. Así, una narrativa única conecta Core, patrones, fallos y métricas.

En la escala de 5,000 documentos, la reconstrucción presentó p95 de 50,859 ms en LokiJS y 521,530 ms en PouchDB; sus intervalos bootstrap de la mediana fueron [45,260; 46,864] ms y [495,701; 502,708] ms, respectivamente. Estas distribuciones confirman el crecimiento del costo de recuperación y evitan reducir el análisis a una sola mediana.

6. Líneas base y experimentos siguientes

El protocolo 1.1 congela controles de CRUD directo, proyección no gobernada, CQRS mínimo, bitácora de eventos y COBRIA C2. Se ejecutaron 450 corridas funcionales sobre LokiJS (cinco controles, tres

escalas y treinta repeticiones). Su finalidad es comprobar instrumentación y costos relativos dentro del mismo arnés; no representan implementaciones industriales completas de CQRS o Event Sourcing.

También se ejecutaron treinta repeticiones de D1–D8: eliminación de la proyección, interrupción, concurrencia, revisiones conflictivas, desconexión, reanudación, migración de esquema y migración entre dos instancias de control. Todos los escenarios funcionales superaron sus aserciones. D5 utiliza desconexión lógica local y D8 dos instancias LokiJS; por ello no constituyen evidencia de partición de red ni migración MongoDB–CouchDB.

7. Banco de fallos y recursos

COBRIA 1.2 añadió diez fallos reproducibles: eliminación total, corrupción parcial, entrega duplicada, revisión fuera de orden, interrupción al 25 %, 50 % y 75 %, cambio de esquema, catálogo inválido y fuga de ámbito. Se ejecutaron treinta repeticiones de cada escenario, 300 en total. La primera ejecución reveló que la migración de esquema no estaba representada; se añadió una transformación explícita y la batería completa terminó con 30/30 aceptaciones por escenario. Este hallazgo ilustra el valor del banco: descubrir defectos del arnés antes de atribuir conformidad.

Se realizaron además treinta corridas de concurrencia local con 1,000 escrituras iniciales, cien actualizaciones concurrentes y reconstrucción simultánea. Todas conservaron 1,000 canónicos, 1,000 candidatos y cien revisiones nuevas. La mediana observada del incremento de heap fue 2,708,144 bytes; CPU de usuario, 12,471 microsegundos; y CPU de sistema, 548 microsegundos. Son métricas de un proceso LokiJS, no de un clúster.

8. Revisión de literatura reproducible

Se congeló un protocolo para ACM Digital Library, IEEE Xplore, SpringerLink, USENIX, arXiv identificado como prepublicación y documentación oficial. La cadena combina vistas materializadas, CQRS, Event Sourcing, procedencia, contratos de datos, reconstrucción, reproducibilidad, versiones y multiinquilinato. La inclusión exige trabajo primario o documentación oficial y un mecanismo suficientemente descrito; la exclusión registra duplicados y contenido sin detalle técnico. La revisión completa necesitará doble cribado independiente, por lo que el artículo no la presenta todavía como revisión sistemática concluida.

9. Transferencia y posibilidad de refutación

El protocolo externo propone seis participantes en tres equipos. Cada equipo implementará C2 a partir de la especificación sin explicación del autor. Se medirán tiempo, requisitos aprobados, preguntas, errores de interpretación, pruebas fallidas y divergencia entre implementaciones. La hipótesis requiere al menos 80 % de requisitos y conservará resultados negativos. El estudio incorpora consentimiento, anonimización y retiro voluntario, pero sigue pendiente de personas reales y revisión ética apropiada.

10. Réplica distribuida

Se publicaron adaptadores para MongoDB, CouchDB y OpenSearch y una composición reproducible con volúmenes separados. MongoDB representa almacenamiento documental; CouchDB, documentos con revisiones; OpenSearch, un derivado especializado de búsqueda. MongoDB 8.0, CouchDB 3.4 y OpenSearch 2.19.3 completaron noventa corridas cada uno. OpenSearch 3.2.0 sufrió un fallo de la JVM Java 24 bajo emulación x86_64. El intento se conserva como incidente de plataforma y no se transforma en resultado negativo de COBRIA. Una VM ARM nativa no pudo descargarse por resolución DNS; por ello se usó 2.19.3 con Java 21 como control de compatibilidad y sus cifras no se atribuyen a 3.2.0.

11. Resultados

Las seis celdas superaron conjuntamente P1, P2, R1, S1 y A1. No hubo documentos recuperados desde un ámbito extranjero. La actualización canónica y proyectada produjo una amplificación de dos escrituras. R1 reconstruyó el conjunto completo y una repetición produjo la misma huella.

Cuadro 3: Medianas de las treinta corridas por celda, en milisegundos.

Motor	<i>n</i>	Canónica	Proyección	R1	Conformidad
PouchDB	100	1,186	0,440	8,376	sí
PouchDB	1,000	13,589	4,719	89,301	sí
PouchDB	5,000	69,291	23,131	500,070	sí
LokiJS	100	0,145	0,052	0,663	sí
LokiJS	1,000	1,263	0,409	6,452	sí
LokiJS	5,000	7,026	2,220	46,011	sí

La proyección fue más rápida en las seis celdas, pero esa mejora no es gratuita: cada cambio proyectado añadió una segunda escritura y toda proyección exige reconstrucción, verificación y operación. Las cifras no son parámetros universales. El hallazgo principal es que la misma batería puede medir beneficio y obligaciones de seguridad como una sola decisión.

Cuadro 4: Medianas distribuidas de treinta corridas por celda, en milisegundos.

Motor	<i>n</i>	Canónica	Proyección	R1	Conformidad
MongoDB 8.0	100	8,749	7,815	83,412	sí
MongoDB 8.0	1.000	39,910	34,431	653,424	sí
MongoDB 8.0	5.000	157,856	129,272	3.179,603	sí
CouchDB 3.4	100	98,830	54,119	296,230	sí
CouchDB 3.4	1.000	1.024,590	440,260	2.985,222	sí
CouchDB 3.4	5.000	4.298,485	1.477,093	12.844,488	sí
OpenSearch 2.19.3	100	178,381	167,058	511,445	sí
OpenSearch 2.19.3	1.000	200,557	168,733	851,417	sí
OpenSearch 2.19.3	5.000	450,837	250,173	2.998,821	sí

Las 270 corridas distribuidas conservaron amplificación de dos escrituras y cero documentos cruzados. Las diferencias temporales no constituyen una clasificación general de motores: el adaptador, la API, la emulación, la JVM, la caché y el estado del host forman parte del tratamiento observado.

12. Analítica e inteligencia artificial

Un conjunto de datos reconstruible es una proyección fechada. Declara población, ventana, transformación, catálogos, exclusiones, semilla, huellas y partición temporal. A1 exige que el conjunto pueda rehacerse desde el manifiesto y que el sistema se abstenga si falta evidencia.

En recuperación aumentada, el ámbito debe limitar candidatos antes de similitud, caché o ensamblado de contexto. Cada fragmento conserva fuente, revisión y versión. Estas obligaciones no garantizan exactitud del modelo, pero hacen reproducible el insumo, reducen fugas y permiten retirar productos afectados.

Como prueba funcional delimitada se ejecutaron 30 repeticiones de recuperación léxica determinista. Las treinta reprodujeron el manifiesto, conservaron la partición temporal, produjeron cero documentos de ámbito ajeno, mantuvieron citas válidas y se abstuvieron ante una consulta sin evidencia. Este control prueba el gobierno del insumo; no mide calidad generativa ni representa un modelo de incrustaciones externo.

13. Discusión

La reconstruibilidad cambia la naturaleza de una copia: deja de ser una autoridad protegida manualmente y se convierte en un producto regenerable con contrato. Los resultados apoyan HN1–HN4 para la implementación de referencia. HN5 recibe apoyo condicionado dentro del arnés embebido porque la proyección redujo lectura, pero su costo depende de mezcla de cargas, concurrencia, red y facturación.

COBRIA también permite decidir no crear una proyección. Si un índice canónico satisface la consulta, la representación adicional aumenta superficie de fallo sin beneficio suficiente. El criterio de retiro forma parte del diseño, no de una limpieza posterior.

14. Amenazas a la validez

PouchDB y LokiJS son motores embebidos y la réplica distribuida usa nodos únicos locales; ninguno reproduce regiones, cuotas, replicación administrada, latencia WAN o cobro. Los documentos son sintéticos y pueden omitir irregularidades históricas. Una sola máquina y un solo autor limitan validez externa. OpenSearch se evaluó en 2.19.3, no en 3.2.0. Las pruebas demuestran conformidad del arnés, no seguridad exhaustiva ni comprensión humana. Se necesita réplica independiente, servicios administrados y estudio con desarrolladores externos.

SQLite y PostgreSQL se preservan únicamente como controles históricos del desarrollo del arnés. Sus cifras quedan excluidas de esta evaluación y de cualquier inferencia NoSQL.

Las líneas base son controles funcionales escritos para el mismo arnés y no sustituyen marcos industriales completos. La inyección de fallos no representa corrupción física, regiones o consenso. Las métricas de memoria de JavaScript pueden incluir recolección de basura. El caso ambiental es sintético. La revisión de literatura no cuenta todavía con doble selección y el estudio de transferencia no tiene participantes.

15. Conclusiones

COBRIA ofrece una especificación refutable para gobernar derivados documentales mediante autoridad, ámbito, versión, equivalencia, reconstrucción y operación. Cinco adaptadores evaluados y 450 corridas confirmatorias —180 embebidas y 270 distribuidas— muestran que P1, P2, R1, S1 y A1 pueden ejecutarse como una batería común. La proyección redujo la mediana de lectura en todas las celdas, con amplificación de dos escrituras y reconstrucción creciente con la escala. Las 300 inyecciones de fallos descubrieron una omisión real en la migración de esquema antes de alcanzar 30/30 aceptaciones en cada escenario. Core 1.2 mejora refutabilidad, trazabilidad, pedagogía y reproducibilidad, manteniendo separados resultados confirmatorios, prepilotos e incidentes de plataforma.

La contribución no es una promesa de superioridad universal, sino una forma de hacer explícito qué copia existe, por qué existe, cómo se verifica y cómo se elimina. El siguiente paso empírico es replicar la batería en servicios documentales administrados, repetir OpenSearch 3.2.0 sobre ARM nativo y someter especificación, código y afirmaciones a revisión externa.

Referencias

- [1] Buneman, P., Khanna, S. y Tan, W.-C. (2001). Why and where: A characterization of data provenance. *ICDT*.
- [2] Chang, F. et al. (2006). Bigtable: A distributed storage system for structured data. *OSDI*.
- [3] DeCandia, G. et al. (2007). Dynamo: Amazon’s highly available key-value store. *SOSP*.
- [4] Fowler, M. (2005). CQRS. *martinfowler.com*.
- [5] Gupta, A. y Mumick, I. S. (1993). Maintenance of materialized views: Problems, techniques, and applications. *IEEE Data Engineering Bulletin*.

- [6] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- [7] MongoDB. (2026). Build a Multi-Tenant Architecture. *MongoDB Documentation*.